JADAVPUR UNIVERSITY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Assignment 5 Report

Packet Tracer and Traffic Analysis with Wireshark

Live Packet Capture, Protocol Stack Dissection, ARP Resolution, ICMP Diagnostics, HTTP/TCP Session Flow, Ethernet OUI Identification, and Hierarchy Profiling

Course: Computer Networks Laboratory (BCSE-III)

Student Name: Mandeep Shaw

Section: A1

Roll Number: 302510501002

Assignment: Assignment 5 (Packet Tracer and Traffic Analysis with Wireshark)

Date of Submission: November 2026

Contents

1 Overview and Objectives **1**
1.1 Objectives . . . 1

2 Question 1: ICMP and ARP Traffic Analysis **1**
2.1 Problem Statement . . . 1
2.2 Experimental Methodology . . . 1
2.3 Part 1: Address Resolution Protocol (ARP) Analysis . . . 2
2.4 Part 2: Internet Control Message Protocol (ICMP) Echo Sequence . . . 2

3 Question 2: Web Traffic Analysis **3**
3.1 Problem Statement . . . 3
3.2 Part 2a: Protocol Diversity in Unfiltered Packet Listing . . . 3
3.3 Parts 2b & 2c: HTTP Response Time (RTT) and IP Addresses . . . 4
3.3.1 Extracted Packet Timestamps and Calculations . . . 5
3.3.2 Identified Internet Addresses . . . 5
3.4 Parts 2d & 2e: Dissecting HTTP GET and Host Parameter . . . 5

4 Question 3: Hex and ASCII Representations in Packet Bytes Panel **6**
4.1 Problem Statement . . . 6
4.2 Observation and Packet Dissection . . . 6
4.3 Byte Breakdown . . . 6

5 Question 4: First 4 Bytes Hex Value of the Host Parameter **6**
5.1 Problem Statement . . . 6
5.2 Analysis and Hex-to-ASCII Decoding . . . 6

6 Question 5: Protocol Filtering and Following Network Flows **7**
6.1 Problem Statement . . . 7
6.2 Protocol Display Filters in Wireshark . . . 7
6.3 Part 5a: Following Network Conversations (UDP Stream Reassembly) . . . 8
6.3.1 Analysis of Stream 8 Payload . . . 9

7 Question 6: Server HTTP Response and Ethernet Layer Inspection **9**
7.1 Problem Statement . . . 9
7.2 Filtering and Ethernet Layer Dissection . . . 10
7.3 Structural Breakdown of Ethernet II Frame (Packet 224) . . . 10

8 Question 7: NIC Manufacturers of PC and Server **10**
8.1 Problem Statement . . . 10
8.2 Manufacturer Identification and Networking Principles . . . 11
8.2.1 Critical Theoretical Insight: Hop-by-Hop MAC Scope . . . 11

9 Question 8: Organizationally Unique Identifiers (OUIs) in Raw Hex Bytes **11**
9.1 Problem Statement . . . 11
9.2 Theoretical Background . . . 11
9.3 Inspection in Packet Bytes Panel . . . 11
9.4 Extracted OUI Hex Values . . . 12

10 Question 9: Protocol Hierarchy Statistics **12**
10.1 Problem Statement . . . 12
10.2 Aggregate Capture Hierarchy Profiling . . . 12
10.3 Part 9a: TCP Statistics and Application Protocols . . . 13
10.4 Part 9b: UDP Statistics and Application Protocols . . . 14
10.5 Comparative Analysis: Packet Count vs. Byte Volume Asymmetry . . . 14

11 Question 10: Traffic Flow Graph Analysis **14**
11.1 Problem Statement . . . 14
11.2 Overview of Wireshark Flow Graph . . . 14
11.3 Step 1: General Flow Graph (All Flows, Network Source) . . . 14
11.3.1 General Flow Observations: . . . 15
11.4 Step 2: TCP Flow Graph (TCP Connection State Machine) . . . 15
11.4.1 Detailed Breakdown of the TCP Flow Lifecycle . . . 16

12 Conclusion and Comments **16**
12.1 Summary of Results . . . 16
12.2 What I Learned . . . 16

1 Overview and Objectives

Network protocol analysis and deep packet inspection (DPI) are fundamental techniques for network diagnostic debugging, performance profiling, cybersecurity audits, and empirical verification of protocol specifications. **Wireshark** is an industry-standard cross-platform packet capture and network protocol analysis suite built on top of the `libpcap`/`dumpcap` capture library.

Wireshark partitions captured network traffic into three interconnected interactive inspection panes:

1. **Packet List Panel:** Displays a chronological summary of each captured frame with columns for packet index number, relative or absolute timestamp, source and destination network/hardware addresses, transport/application protocol identification, length in octets, and protocol-specific summary metadata.
2. **Packet Details Panel:** Presents an expandable hierarchical tree dissecting the protocol stack layer-by-layer (Frame → Ethernet II → IPv4/IPv6 → TCP/UDP/ICMP → Application layer protocols such as HTTP, DNS, or mDNS).
3. **Packet Bytes Panel:** Displays the raw byte buffer of the selected packet in traditional hex dump format (16 octets per row in hexadecimal) alongside corresponding ASCII character encodings, with bidirectional selection synchronization linked to the details tree.

1.1 Objectives

The primary objectives of this laboratory assignment are:

- Capture live network traffic across physical and wireless interfaces using Wireshark and command-line utilities.
- Analyze link-layer **Address Resolution Protocol (ARP)** request broadcasts and unicast replies used to map IP addresses to physical MAC addresses.
- Investigate **Internet Control Message Protocol (ICMP)** Echo Request and Echo Reply messages generated by network diagnostic tools (`ping`).
- Inspect application-layer **Hypertext Transfer Protocol (HTTP)** requests and responses, calculating precise end-to-end Round-Trip Time (RTT) and identifying client/server IP addresses.
- Examine packet bytes in raw hexadecimal and ASCII format, verifying field alignments and identifying specific HTTP header tokens.
- Master Wireshark display filters to isolate traffic across protocols (`http`, `tcp`, `dns`, `udp`) and follow application conversation flows (Stream Reassembly).
- Deconstruct Ethernet II frame headers, distinguish between local client MAC addresses and hop-by-hop default gateway MAC addresses, and decode IEEE 802 **Organizationally Unique Identifiers (OUIs)**.
- Generate protocol hierarchy statistics to profile packet and byte distributions across transport layers (TCP vs. UDP).
- Construct temporal ladder flow graphs to trace connection establishment (three-way handshake), data transfer, and graceful termination (four-way handshake).

2 Question 1: ICMP and ARP Traffic Analysis

2.1 Problem Statement

Generate some ICMP traffic by using the Ping command line tool to check the connectivity of a neighbouring machine (or router). Note the results in Wireshark. The initial ARP request broadcast from your PC determines the physical MAC address of the network IP Address, and the ARP reply from the neighboring system. After the ARP request, the pings (ICMP echo request and replies) can be seen.

2.2 Experimental Methodology

To examine the interaction between link-layer address resolution and network-layer reachability testing:

1. The local network host was configured on the subnet with IP address `10.223.117.129` and MAC address `ce:7e:13:24:96:68`.
2. A live packet capture was initiated in Wireshark on the network interface `enxce7e13249668`.

3. The `ping` utility was invoked to ping a neighbouring machine at IP `10.223.117.59`:
```
$ ping -c 4 10.223.117.59
```
4. Wireshark display filters `arp` and `icmp` were applied respectively to isolate address resolution and ICMP signaling.

2.3 Part 1: Address Resolution Protocol (ARP) Analysis

Before an IP datagram can be transmitted over a local Ethernet/802.3 broadcast domain, the host's operating system consults its internal ARP cache. Because no entry initially exists for target IP `10.223.117.59`, an ARP Request frame is broadcast to the Layer-2 broadcast address `ff:ff:ff:ff:ff:ff`.

arp

No.	Time	Source	Destination	Protocol	Length	Info
359	74.938207335	ce:7e:13:24:96:68	46:73:bf:37:e8:7f	ARP	42	Who has 10.223.117.59? Tell 10.223.117.129
360	74.939520501	46:73:bf:37:e8:7f	ce:7e:13:24:96:68	ARP	42	10.223.117.59 is at 46:73:bf:37:e8:7f
361	75.012304205	46:73:bf:37:e8:7f	ce:7e:13:24:96:68	ARP	42	Who has 10.223.117.129? Tell 10.223.117.59
362	75.012349325	ce:7e:13:24:96:68	46:73:bf:37:e8:7f	ARP	42	10.223.117.129 is at ce:7e:13:24:96:68
476	118.969219799	ce:7e:13:24:96:68	46:73:bf:37:e8:7f	ARP	42	Who has 10.223.117.59? Tell 10.223.117.129
477	118.970840157	46:73:bf:37:e8:7f	ce:7e:13:24:96:68	ARP	42	10.223.117.59 is at 46:73:bf:37:e8:7f
478	119.050267731	46:73:bf:37:e8:7f	ce:7e:13:24:96:68	ARP	42	Who has 10.223.117.129? Tell 10.223.117.59
479	119.050294485	ce:7e:13:24:96:68	46:73:bf:37:e8:7f	ARP	42	10.223.117.129 is at ce:7e:13:24:96:68
667	146.702839974	46:73:bf:37:e8:7f	ce:7e:13:24:96:68	ARP	42	Who has 10.223.117.129? Tell 10.223.117.59
668	146.702858692	ce:7e:13:24:96:68	46:73:bf:37:e8:7f	ARP	42	10.223.117.129 is at ce:7e:13:24:96:68
1539	168.121223785	ce:7e:13:24:96:68	46:73:bf:37:e8:7f	ARP	42	Who has 10.223.117.59? Tell 10.223.117.129
1540	168.122454104	46:73:bf:37:e8:7f	ce:7e:13:24:96:68	ARP	42	10.223.117.59 is at 46:73:bf:37:e8:7f
1694	175.633331954	46:73:bf:37:e8:7f	ce:7e:13:24:96:68	ARP	42	Who has 10.223.117.129? Tell 10.223.117.59
1695	175.633359195	ce:7e:13:24:96:68	46:73:bf:37:e8:7f	ARP	42	10.223.117.129 is at ce:7e:13:24:96:68
3360	201.748982399	46:73:bf:37:e8:7f	ce:7e:13:24:96:68	ARP	42	Who has 10.223.117.129? Tell 10.223.117.59
3361	201.749012925	ce:7e:13:24:96:68	46:73:bf:37:e8:7f	ARP	42	10.223.117.129 is at ce:7e:13:24:96:68
3650	249.529204818	ce:7e:13:24:96:68	46:73:bf:37:e8:7f	ARP	42	Who has 10.223.117.59? Tell 10.223.117.129
3651	249.530597019	46:73:bf:37:e8:7f	ce:7e:13:24:96:68	ARP	42	10.223.117.59 is at 46:73:bf:37:e8:7f
3652	249.628707679	46:73:bf:37:e8:7f	ce:7e:13:24:96:68	ARP	42	Who has 10.223.117.129? Tell 10.223.117.59
3653	249.628746727	ce:7e:13:24:96:68	46:73:bf:37:e8:7f	ARP	42	10.223.117.129 is at ce:7e:13:24:96:68
3976	306.215200702	46:73:bf:37:e8:7f	ce:7e:13:24:96:68	ARP	42	Who has 10.223.117.129? Tell 10.223.117.59
3977	306.215225082	ce:7e:13:24:96:68	46:73:bf:37:e8:7f	ARP	42	10.223.117.129 is at ce:7e:13:24:96:68
4639	330.028312342	46:73:bf:37:e8:7f	ce:7e:13:24:96:68	ARP	42	Who has 10.223.117.129? Tell 10.223.117.59
4640	330.028341891	ce:7e:13:24:96:68	46:73:bf:37:e8:7f	ARP	42	10.223.117.129 is at ce:7e:13:24:96:68
5228	339.130276103	ce:7e:13:24:96:68	46:73:bf:37:e8:7f	ARP	42	Who has 10.223.117.59? Tell 10.223.117.129
5229	339.131652548	46:73:bf:37:e8:7f	ce:7e:13:24:96:68	ARP	42	10.223.117.59 is at 46:73:bf:37:e8:7f
5406	355.377705355	46:73:bf:37:e8:7f	ce:7e:13:24:96:68	ARP	42	Who has 10.223.117.129? Tell 10.223.117.59
5407	355.377749714	ce:7e:13:24:96:68	46:73:bf:37:e8:7f	ARP	42	10.223.117.129 is at ce:7e:13:24:96:68
5881	480.953212951	ce:7e:13:24:96:68	46:73:bf:37:e8:7f	ARP	42	Who has 10.223.117.59? Tell 10.223.117.129
5882	480.954538880	46:73:bf:37:e8:7f	ce:7e:13:24:96:68	ARP	42	10.223.117.59 is at 46:73:bf:37:e8:7f
5883	481.027907621	46:73:bf:37:e8:7f	ce:7e:13:24:96:68	ARP	42	Who has 10.223.117.129? Tell 10.223.117.59
5884	481.027944138	ce:7e:13:24:96:68	46:73:bf:37:e8:7f	ARP	42	10.223.117.129 is at ce:7e:13:24:96:68
6746	499.711447985	46:73:bf:37:e8:7f	ce:7e:13:24:96:68	ARP	42	Who has 10.223.117.129? Tell 10.223.117.59
6747	499.711484924	ce:7e:13:24:96:68	46:73:bf:37:e8:7f	ARP	42	10.223.117.129 is at ce:7e:13:24:96:68
7199	586.744453126	46:73:bf:37:e8:7f	ce:7e:13:24:96:68	ARP	42	Who has 10.223.117.129? Tell 10.223.117.59
7200	586.744492586	ce:7e:13:24:96:68	46:73:bf:37:e8:7f	ARP	42	10.223.117.129 is at ce:7e:13:24:96:68
7201	586.937211911	ce:7e:13:24:96:68	46:73:bf:37:e8:7f	ARP	42	Who has 10.223.117.59? Tell 10.223.117.129
7202	586.938555869	46:73:bf:37:e8:7f	ce:7e:13:24:96:68	ARP	42	10.223.117.59 is at 46:73:bf:37:e8:7f

Figure 1: Wireshark packet capture filtered by `arp`, capturing ARP request broadcasts and corresponding unicast replies between `10.223.117.129` and `10.223.117.59`.

As captured in Figure 1:

- **ARP Request (Packet No. 5228):**
  - **Source MAC:** `ce:7e:13:24:96:68` (Transmitting PC)
  - **Destination MAC:** `ff:ff:ff:ff:ff:ff` (Ethernet Broadcast)
  - **Sender IP:** `10.223.117.129` → **Target IP:** `10.223.117.59`
  - **Info:** `Who has 10.223.117.59? Tell 10.223.117.129`
- **ARP Reply (Packet No. 5229):**
  - **Source MAC:** `46:73:bf:37:e8:7f` (Neighbouring Target System)
  - **Destination MAC:** `ce:7e:13:24:96:68` (Unicast reply to transmitting PC)
  - **Sender IP:** `10.223.117.59` → **Target IP:** `10.223.117.129`
  - **Info:** `10.223.117.59 is at 46:73:bf:37:e8:7f`

Once received, the local host updates its kernel ARP routing table, allowing the outbound IP packet to be properly framed with destination hardware address `46:73:bf:37:e8:7f`.

2.4 Part 2: Internet Control Message Protocol (ICMP) Echo Sequence

With the destination MAC address resolved, the diagnostic ping packets are transmitted. The display filter `icmp` captures four consecutive Echo Request and Echo Reply pairs.

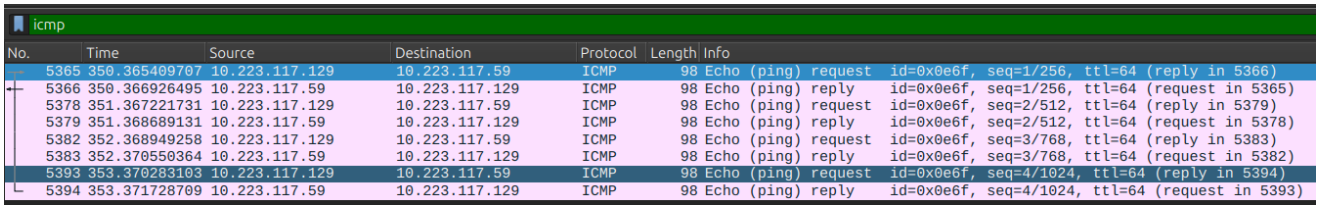


Figure 2: Wireshark packet capture filtered by `icmp`, showing alternating ICMP Echo Requests (Type 8) and Echo Replies (Type 0) with matching process IDs and sequence numbers.

No.	Time (s)	Source IP	Destination IP	Type / Code	Len	ICMP Details
5365	350.365409	10.223.117.129	10.223.117.59	Type 8, Code 0	98	Echo request, id=0x0e6f, seq=1, ttl=64
5366	350.366926	10.223.117.59	10.223.117.129	Type 0, Code 0	98	Echo reply, id=0x0e6f, seq=1, ttl=64
5378	351.367221	10.223.117.129	10.223.117.59	Type 8, Code 0	98	Echo request, id=0x0e6f, seq=2, ttl=64
5379	351.368689	10.223.117.59	10.223.117.129	Type 0, Code 0	98	Echo reply, id=0x0e6f, seq=2, ttl=64
5382	352.368949	10.223.117.129	10.223.117.59	Type 8, Code 0	98	Echo request, id=0x0e6f, seq=3, ttl=64
5383	352.370550	10.223.117.59	10.223.117.129	Type 0, Code 0	98	Echo reply, id=0x0e6f, seq=3, ttl=64
5393	353.370283	10.223.117.129	10.223.117.59	Type 8, Code 0	98	Echo request, id=0x0e6f, seq=4, ttl=64
5394	353.371728	10.223.117.59	10.223.117.129	Type 0, Code 0	98	Echo reply, id=0x0e6f, seq=4, ttl=64

Table 1: Dissected ICMP echo sequence parameters from the captured trace.

Mathematical Latency Evaluation: For sequence 1 (Packets 5365 and 5366):

$$\text{RTT}_1 = T_{\text{reply}} - T_{\text{request}} = 350.366926495 - 350.365409707 = 0.001516788 \text{ seconds} \approx \mathbf{1.517} \text{ ms} \quad (1)$$

The identifier `0x0e6f` (decimal 3695) uniquely identifies the Linux `ping` process instance, ensuring that replies are demultiplexed to the originating terminal session.

3 Question 2: Web Traffic Analysis

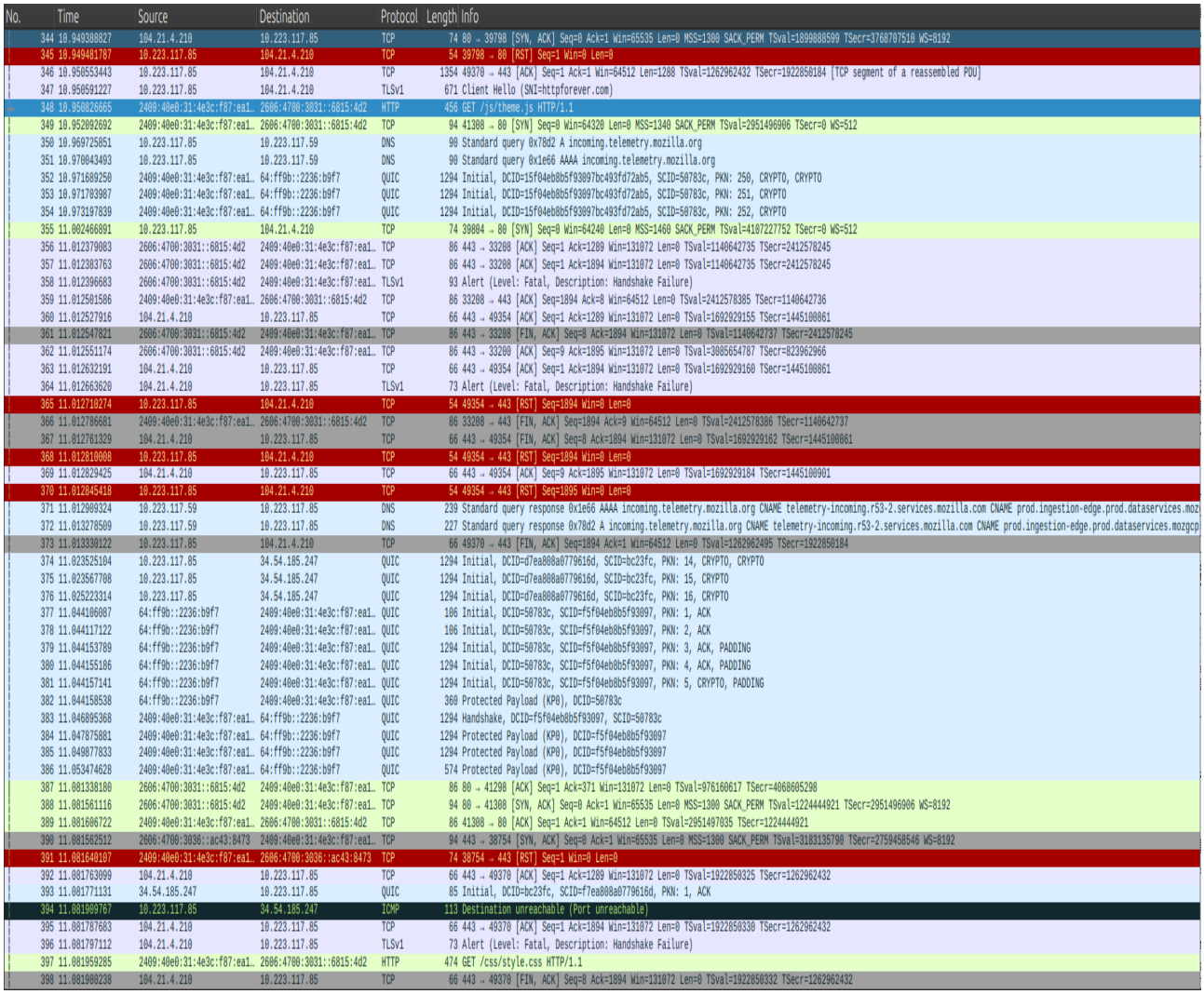
3.1 Problem Statement

Generate some web traffic and:

a. *Find the list of the different protocols that appear in the protocol column in the unfiltered packet-listing window of Wireshark.*
b. *How long did it take from when the HTTP GET message was sent until the HTTP OK reply was received? (By default, the value of the Time column in the packet-listing window is the amount of time, in seconds, since Wireshark tracing began. To display the Time field in time-of-day format, select the Wireshark View pull down menu, then select Time Display Format, then select Time-of-day.)*
c. *What is the Internet address of the website? What is the Internet address of your computer?*
d. *Search back through your capture, and find an HTTP packet containing a GET command. Click on the packet in the Packet List Panel. Then expand the HTTP layer in the Packet Details Panel, from the packet.*
e. *Find out the value of the Host from the Packet Details Panel, within the GET command.*

3.2 Part 2a: Protocol Diversity in Unfiltered Packet Listing

A web browser was launched to generate HTTP/HTTPS traffic. Inspection of the unfiltered packet capture reveals a diverse cross-section of protocols operating across the network architecture.



No.	Time	Source	Destination	Protocol	Length	Info
344	10.949388827	104.21.4.210	10.223.117.85	TCP	74	80 → 39798 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1300 SACK_PERM TSval=1899888599 TSecr=3768707510 WS=8192
345	10.949481787	10.223.117.85	104.21.4.210	TCP	54	39798 → 80 [RST] Seq=1 Win=0 Len=0
346	10.950553443	10.223.117.85	104.21.4.210	TCP	1354	49370 → 443 [ACK] Seq=1 Ack=1 Win=64512 Len=1288 TSval=1262962432 TSecr=1922850184 [TCP segment of a reassembled PDU]
347	10.950591227	10.223.117.85	104.21.4.210	TLSv1	671	Client Hello (SNI=httpforever.com)
348	10.950826665	2409:40e0:31:4e3c:f87:ea1…	2606:4700:3031::6815:4d2	HTTP	456	GET /js/theme.js HTTP/1.1
349	10.952092692	2409:40e0:31:4e3c:f87:ea1…	2606:4700:3031::6815:4d2	TCP	94	41308 → 80 [SYN] Seq=0 Win=64320 Len=0 MSS=1340 SACK_PERM TSval=2951496906 TSecr=0 WS=512
350	10.969725851	10.223.117.85	10.223.117.59	DNS	90	Standard query 0x78d2 A incoming.telemetry.mozilla.org
351	10.970043493	10.223.117.85	10.223.117.59	DNS	90	Standard query 0x1e66 AAAA incoming.telemetry.mozilla.org
352	10.971689250	2409:40e0:31:4e3c:f87:ea1…	64:ff9b::2236:b9f7	QUIC	1294	Initial, DCID=15f04eb8b5f93097bc493fd72ab5, SCID=50783c, PKN: 250, CRYPTO, CRYPTO
353	10.971783987	2409:40e0:31:4e3c:f87:ea1…	64:ff9b::2236:b9f7	QUIC	1294	Initial, DCID=15f04eb8b5f93097bc493fd72ab5, SCID=50783c, PKN: 251, CRYPTO
354	10.973197839	2409:40e0:31:4e3c:f87:ea1…	64:ff9b::2236:b9f7	QUIC	1294	Initial, DCID=15f04eb8b5f93097bc493fd72ab5, SCID=50783c, PKN: 252, CRYPTO
355	11.002466891	10.223.117.85	104.21.4.210	TCP	74	39804 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=4187227752 TSecr=0 WS=512
356	11.012379083	2606:4700:3031::6815:4d2	2409:40e0:31:4e3c:f87:ea1…	TCP	86	443 → 33208 [ACK] Seq=1 Ack=1289 Win=131072 Len=0 TSval=1140642735 TSecr=2412578245
357	11.012383763	2606:4700:3031::6815:4d2	2409:40e0:31:4e3c:f87:ea1…	TCP	86	443 → 33208 [ACK] Seq=1 Ack=1894 Win=131072 Len=0 TSval=1140642735 TSecr=2412578245
358	11.012396683	2606:4700:3031::6815:4d2	2409:40e0:31:4e3c:f87:ea1…	TLSv1	93	Alert (Level: Fatal, Description: Handshake Failure)
359	11.012501586	2409:40e0:31:4e3c:f87:ea1…	2606:4700:3031::6815:4d2	TCP	86	33208 → 443 [ACK] Seq=1894 Ack=8 Win=64512 Len=0 TSval=2412578385 TSecr=1140642736
360	11.012527916	104.21.4.210	10.223.117.85	TCP	66	443 → 49354 [ACK] Seq=1 Ack=1289 Win=131072 Len=0 TSval=1692929155 TSecr=1445100861
361	11.012547821	2606:4700:3031::6815:4d2	2409:40e0:31:4e3c:f87:ea1…	TCP	86	443 → 33208 [FIN, ACK] Seq=8 Ack=1894 Win=131072 Len=0 TSval=1140642737 TSecr=2412578245
362	11.012551174	2606:4700:3031::6815:4d2	2409:40e0:31:4e3c:f87:ea1…	TCP	86	443 → 33200 [ACK] Seq=9 Ack=1895 Win=131072 Len=0 TSval=3885654787 TSecr=823962966
363	11.012632191	104.21.4.210	10.223.117.85	TCP	66	443 → 49354 [ACK] Seq=1 Ack=1894 Win=131072 Len=0 TSval=1692929160 TSecr=1445100861
364	11.012663620	104.21.4.210	10.223.117.85	TLSv1	73	Alert (Level: Fatal, Description: Handshake Failure)
365	11.012710274	10.223.117.85	104.21.4.210	TCP	54	49354 → 443 [RST] Seq=1894 Win=0 Len=0
366	11.012786681	2409:40e0:31:4e3c:f87:ea1…	2606:4700:3031::6815:4d2	TCP	86	33208 → 443 [FIN, ACK] Seq=1894 Ack=9 Win=64512 Len=0 TSval=2412578386 TSecr=1140642737
367	11.012761329	104.21.4.210	10.223.117.85	TCP	66	443 → 49354 [FIN, ACK] Seq=8 Ack=1894 Win=131072 Len=0 TSval=1692929162 TSecr=1445100861
368	11.012810008	10.223.117.85	104.21.4.210	TCP	54	49354 → 443 [RST] Seq=1894 Win=0 Len=0
369	11.012829425	104.21.4.210	10.223.117.85	TCP	66	443 → 49354 [ACK] Seq=9 Ack=1895 Win=131072 Len=0 TSval=1692929184 TSecr=1445100901
370	11.012845418	10.223.117.85	104.21.4.210	TCP	54	49354 → 443 [RST] Seq=1895 Win=0 Len=0
371	11.012909324	10.223.117.59	10.223.117.85	DNS	239	Standard query response 0x1e66 AAAA incoming.telemetry.mozilla.org CNAME telemetry-incoming.r53-2.services.mozilla.com CNAME prod.ingestion-edge.prod.dataservices.moz
372	11.013278509	10.223.117.59	10.223.117.85	DNS	227	Standard query response 0x78d2 A incoming.telemetry.mozilla.org CNAME telemetry-incoming.r53-2.services.mozilla.com CNAME prod.ingestion-edge.prod.dataservices.mozgcp
373	11.013330122	10.223.117.85	104.21.4.210	TCP	66	49370 → 443 [FIN, ACK] Seq=1894 Ack=1 Win=64512 Len=0 TSval=1262962495 TSecr=1922850184
374	11.023525104	10.223.117.85	34.54.185.247	QUIC	1294	Initial, DCID=d7ea808a0779616d, SCID=bc23fc, PKN: 14, CRYPTO, CRYPTO
375	11.023567708	10.223.117.85	34.54.185.247	QUIC	1294	Initial, DCID=d7ea808a0779616d, SCID=bc23fc, PKN: 15, CRYPTO
376	11.025223314	10.223.117.85	34.54.185.247	QUIC	1294	Initial, DCID=d7ea808a0779616d, SCID=bc23fc, PKN: 16, CRYPTO
377	11.044106087	64:ff9b::2236:b9f7	2409:40e0:31:4e3c:f87:ea1…	QUIC	106	Initial, DCID=50783c, SCID=f5f04eb8b5f93097, PKN: 1, ACK
378	11.044117122	64:ff9b::2236:b9f7	2409:40e0:31:4e3c:f87:ea1…	QUIC	106	Initial, DCID=50783c, SCID=f5f04eb8b5f93097, PKN: 2, ACK
379	11.044153789	64:ff9b::2236:b9f7	2409:40e0:31:4e3c:f87:ea1…	QUIC	1294	Initial, DCID=50783c, SCID=f5f04eb8b5f93097, PKN: 3, ACK, PADDING
380	11.044155186	64:ff9b::2236:b9f7	2409:40e0:31:4e3c:f87:ea1…	QUIC	1294	Initial, DCID=50783c, SCID=f5f04eb8b5f93097, PKN: 4, ACK, PADDING
381	11.044157141	64:ff9b::2236:b9f7	2409:40e0:31:4e3c:f87:ea1…	QUIC	1294	Initial, DCID=50783c, SCID=f5f04eb8b5f93097, PKN: 5, CRYPTO, PADDING
382	11.044158538	64:ff9b::2236:b9f7	2409:40e0:31:4e3c:f87:ea1…	QUIC	360	Protected Payload (KP0), DCID=50783c
383	11.046895368	2409:40e0:31:4e3c:f87:ea1…	64:ff9b::2236:b9f7	QUIC	1294	Handshake, DCID=f5f04eb8b5f93097, SCID=50783c
384	11.047875881	2409:40e0:31:4e3c:f87:ea1…	64:ff9b::2236:b9f7	QUIC	1294	Protected Payload (KP0), DCID=f5f04eb8b5f93097
385	11.049877833	2409:40e0:31:4e3c:f87:ea1…	64:ff9b::2236:b9f7	QUIC	1294	Protected Payload (KP0), DCID=f5f04eb8b5f93097
386	11.053474628	2409:40e0:31:4e3c:f87:ea1…	64:ff9b::2236:b9f7	QUIC	574	Protected Payload (KP0), DCID=f5f04eb8b5f93097
387	11.081338180	2606:4700:3031::6815:4d2	2409:40e0:31:4e3c:f87:ea1…	TCP	86	80 → 41298 [ACK] Seq=1 Ack=371 Win=131072 Len=0 TSval=976160617 TSecr=4068605298
388	11.081561116	2606:4700:3031::6815:4d2	2409:40e0:31:4e3c:f87:ea1…	TCP	94	80 → 41308 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1300 SACK_PERM TSval=1224444921 TSecr=2951496906 WS=8192
389	11.081606722	2409:40e0:31:4e3c:f87:ea1…	2606:4700:3031::6815:4d2	TCP	86	41308 → 80 [ACK] Seq=1 Ack=1 Win=64512 Len=0 TSval=2951497035 TSecr=1224444921
390	11.081562512	2606:4700:3036::ac43:8473	2409:40e0:31:4e3c:f87:ea1…	TCP	94	443 → 38754 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1300 SACK_PERM TSval=3183135790 TSecr=2759458546 WS=8192
391	11.081640107	2409:40e0:31:4e3c:f87:ea1…	2606:4700:3036::ac43:8473	TCP	74	38754 → 443 [RST] Seq=1 Win=0 Len=0
392	11.081763099	104.21.4.210	10.223.117.85	TCP	66	443 → 49370 [ACK] Seq=1 Ack=1289 Win=131072 Len=0 TSval=1922850325 TSecr=1262962432
393	11.081771131	34.54.185.247	10.223.117.85	QUIC	85	Initial, DCID=bc23fc, SCID=f7ea808a0779616d, PKN: 1, ACK
394	11.081909767	10.223.117.85	34.54.185.247	ICMP	113	Destination unreachable (Port unreachable)
395	11.081787683	104.21.4.210	10.223.117.85	TCP	66	443 → 49370 [ACK] Seq=1 Ack=1894 Win=131072 Len=0 TSval=1922850330 TSecr=1262962432
396	11.081797112	104.21.4.210	10.223.117.85	TLSv1	73	Alert (Level: Fatal, Description: Handshake Failure)
397	11.081959285	2409:40e0:31:4e3c:f87:ea1…	2606:4700:3031::6815:4d2	HTTP	474	GET /css/style.css HTTP/1.1
398	11.081988238	104.21.4.210	10.223.117.85	TCP	66	443 → 49370 [FIN, ACK] Seq=8 Ack=1894 Win=131072 Len=0 TSval=1922850332 TSecr=1262962432

Figure 3: Unfiltered packet capture displaying concurrent interleaved protocols during web browsing.

The distinct protocols observed in the **Protocol** column are:

1. **TCP (Transmission Control Protocol):** Provides connection-oriented, reliable byte-stream transmission, handling the 3-way handshake (`SYN`, `SYN-ACK`, `ACK`), acknowledgment packets, and teardown (`FIN`, `RST`).
2. **TLSv1 / TLSv1.2 (Transport Layer Security):** Provides cryptographic privacy and data integrity for secure web traffic, visible during client hello negotiations, cipher exchanges, and encrypted application records.
3. **HTTP (Hypertext Transfer Protocol):** The cleartext application-layer protocol used to request and transfer web assets (visible in requests such as `GET /js/theme.js HTTP/1.1` and `GET /css/style.css HTTP/1.1`).
4. **DNS (Domain Name System):** Resolves fully qualified domain names (FQDNs) to IPv4/IPv6 addresses (e.g., standard queries for `incoming.telemetry.mozilla.org`).
5. **QUIC (Quick UDP Internet Connections):** Modern multiplexed transport protocol operating on top of UDP port 443, utilized by modern web browsers for low-latency encrypted sessions.
6. **ICMP (Internet Control Message Protocol):** Observed carrying control messages such as `Destination unreachable (Port unreachable)` generated when UDP probe sockets are closed.

3.3 Parts 2b & 2c: HTTP Response Time (RTT) and IP Addresses

The Wireshark time display format was changed to **Time-of-day** via `View` → `Time Display Format` → `Time-of-day`. The capture was filtered with the display filter `http` to isolate the HTTP request-response transaction.

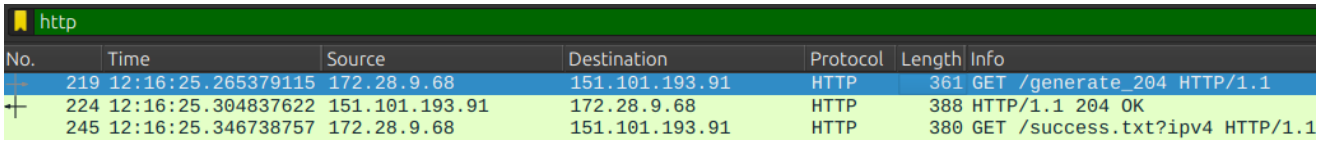


Figure 4: HTTP GET request (Packet 219) and matching HTTP 204 OK response (Packet 224) displayed in Time-of-day format.

3.3.1 Extracted Packet Timestamps and Calculations

- **HTTP GET Request (Packet No. 219):**
  - Timestamp: `12:16:25.265379115`
  - Source IP: `172.28.9.68` (Client Machine)
  - Destination IP: `151.101.193.91` (Web Server)
  - Request Line: `GET /generate_204 HTTP/1.1`
- **HTTP OK Reply (Packet No. 224):**
  - Timestamp: `12:16:25.304837622`
  - Source IP: `151.101.193.91` (Web Server)
  - Destination IP: `172.28.9.68` (Client Machine)
  - Status Code: `HTTP/1.1 204 OK` (No Content)

Mathematical Computation of Elapsed Response Time:

$$\begin{aligned} \Delta T &= T_{\text{reply}} - T_{\text{request}} \\ \Delta T &= 12:16:25.304837622 - 12:16:25.265379115 \\ \Delta T &= 0.039458507 \text{ seconds} \approx \mathbf{39.46} \text{ milliseconds} \end{aligned} \tag{2}$$

The measured round-trip time of 39.46 ms encompasses network propagation delay, routing latency, TCP ACK handling, and server processing.

3.3.2 Identified Internet Addresses

- **Internet Address of the Website / Server:** `151.101.193.91` (Fastly Global CDN IP serving the host domain).
- **Internet Address of My Computer (Client):** `172.28.9.68` (Local private IPv4 address assigned to wireless NIC `wlp1s0`).

3.4 Parts 2d & 2e: Dissecting HTTP GET and Host Parameter

Selecting Packet 219 in the Packet List Panel and expanding the **Hypertext Transfer Protocol** layer in the Packet Details Panel reveals the complete set of HTTP request headers.

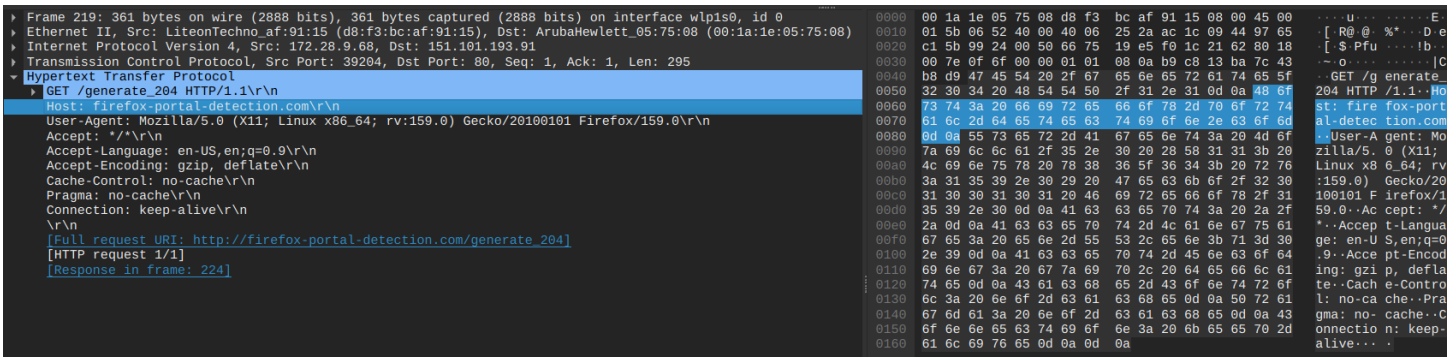


Figure 5: Expanded HTTP layer in the Packet Details Panel for Packet 219, highlighting the `Host` parameter and its corresponding raw hex and ASCII byte representation.

Expanded Protocol Header Structure:

- **Request Method:** `GET`
- **Request URI:** `/generate_204`
- **Request Version:** `HTTP/1.1`

- **Host Parameter Line:** Host: firefox −portal−detection.com\r\n
- **User-Agent:** `Mozilla/5.0 (X11; Linux x86_64; rv:159.0) Gecko/20100101 Firefox/159.0`
- **Connection:** `keep-alive`

Answer for 2e: The extracted value of the Host parameter is `firefox-portal-detection.com`.

4 Question 3: Hex and ASCII Representations in Packet Bytes Panel

4.1 Problem Statement

Highlight the Hex and ASCII representations of the packet in the Packet Bytes Panel.

4.2 Observation and Packet Dissection

In Wireshark, the Packet Details Panel and Packet Bytes Panel maintain bidirectional field-to-byte synchronization. When the Host: firefox −portal−detection.com\r\n field is clicked in the middle Details Panel, the corresponding bytes are immediately highlighted in blue within the bottom-right Packet Bytes Panel, as evidenced in Figure 5.

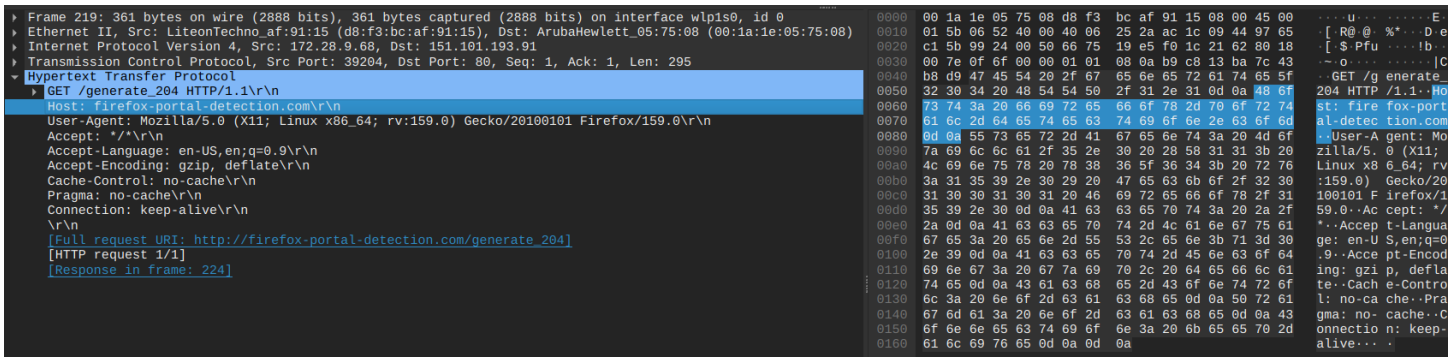


Figure 6: Detailed inspection showing the synchronized highlighting of the `Host` header in both hexadecimal (left) and ASCII (right) formats within the Packet Bytes Panel.

4.3 Byte Breakdown

The exact highlighted slice from offset `0x005e` to `0x0080` contains 35 octets:

- **Highlighted Hexadecimal Stream:**

```
48 6f 73 74 3a 20 66 69 72 65 66 6f 78 2d 70 6f 72 74 61 6c 2d 64 65 74 65 63 74 69 6f 6
    e 2e 63 6f 6d 0d 0a
```

- **Highlighted ASCII Stream:**

```
Host: firefox-portal-detection.com..
```

Notice that the non-printable ASCII control characters `0x0d` (Carriage Return, \r) and `0x0a` (Line Feed, \n) at the end of the HTTP header line are displayed as dots (`..`) in the ASCII text pane, perfectly matching the HTTP/1.1 RFC 2616 header termination specification.

5 Question 4: First 4 Bytes Hex Value of the Host Parameter

5.1 Problem Statement

Find out the first 4 bytes of the Hex value of the Host parameter from the Packet Bytes Panel.

5.2 Analysis and Hex-to-ASCII Decoding

Examining the beginning of the highlighted byte region in the Packet Bytes Panel in Figure 6:
Answer: The first 4 bytes of the Hex value of the Host parameter are:

48 6f 73 74

In ASCII, this sequence translates to the literal protocol keyword `Host`.

Byte Position	Hexadecimal	Binary	Decimal	Decoded ASCII
1st Byte	48	01001000	72	**'H'**
2nd Byte	6f	01101111	111	**'o'**
3rd Byte	73	01110011	115	**'s'**
4th Byte	74	01110100	116	**'t'**

Table 2: Mapping of the first 4 hexadecimal bytes of the Host parameter to ASCII characters.

6 Question 5: Protocol Filtering and Following Network Flows

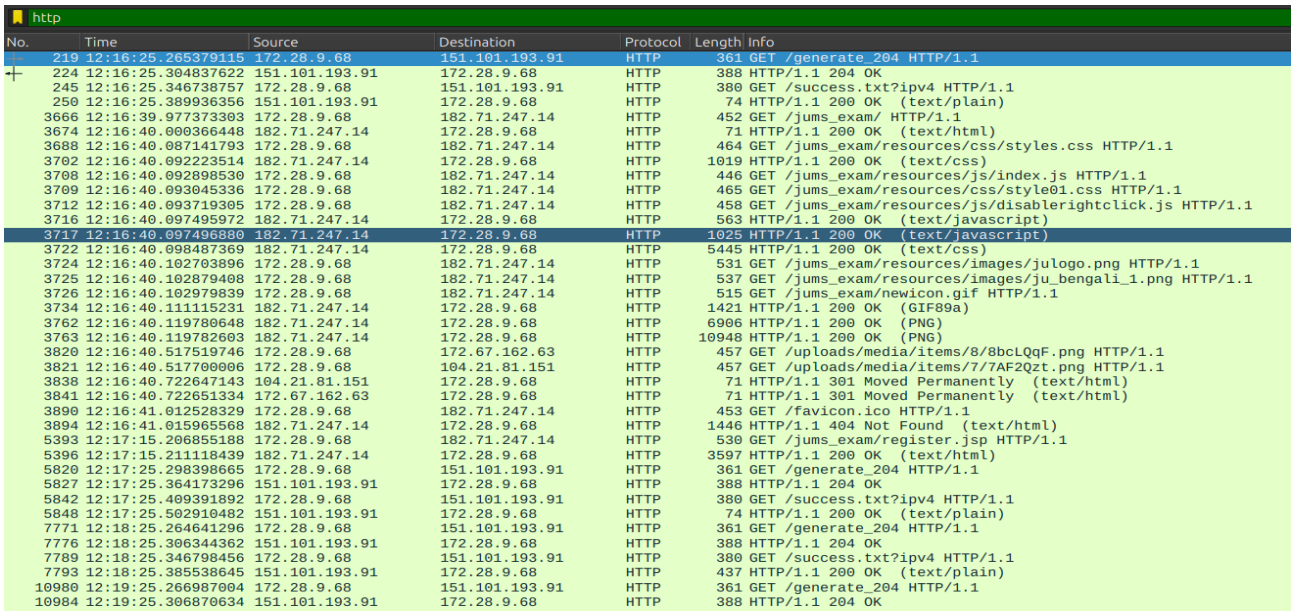
6.1 Problem Statement

Filter packets with http, TCP, DNS and other protocols.

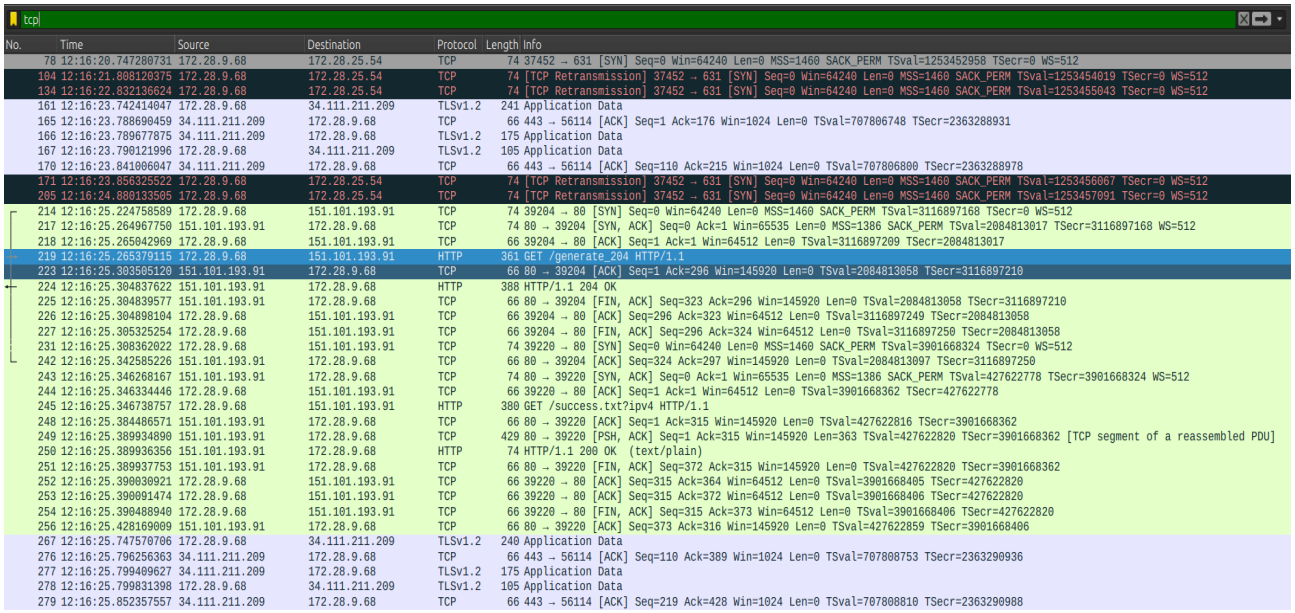
a. *Find out what those packets contain by following one of the conversations (also called network flows), select one of the packets and press the right mouse button..click on follow.*

6.2 Protocol Display Filters in Wireshark

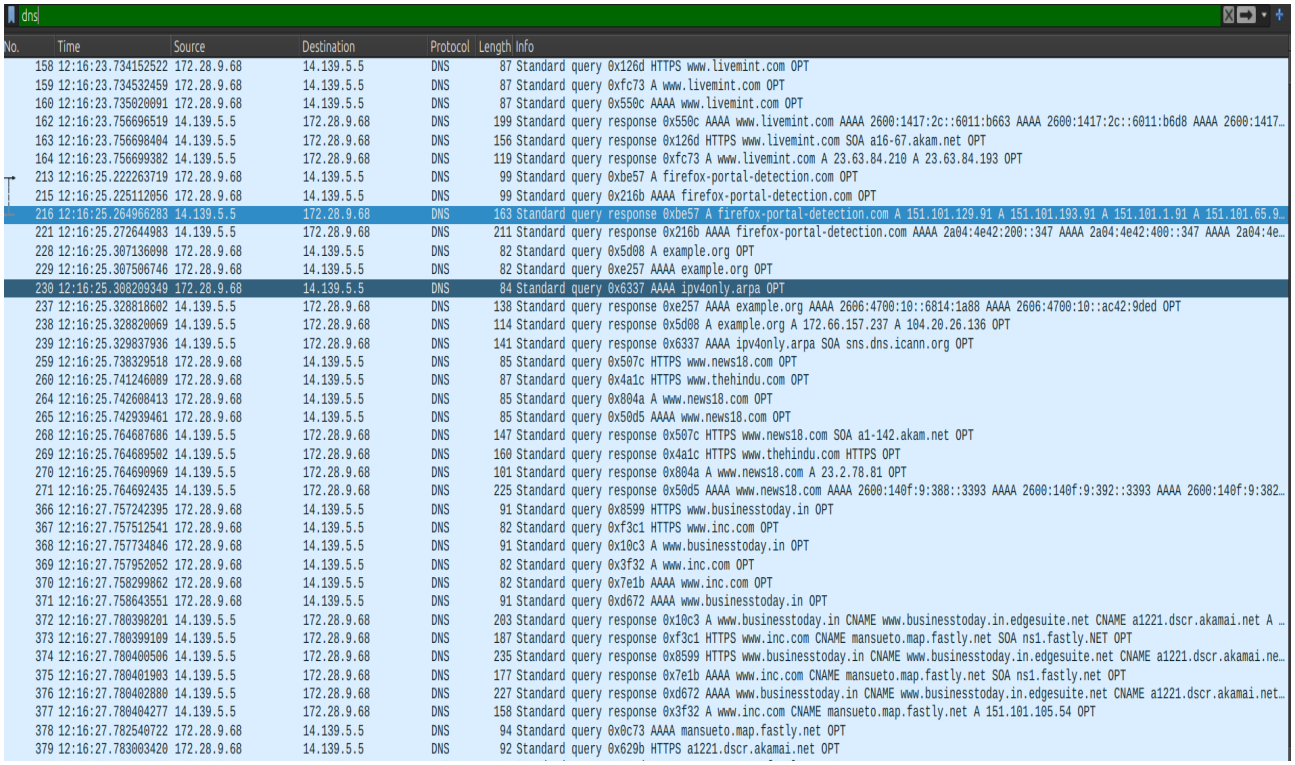
Wireshark provides a boolean expression engine for filtering packet views without altering underlying capture buffers. The following primary filters were evaluated:



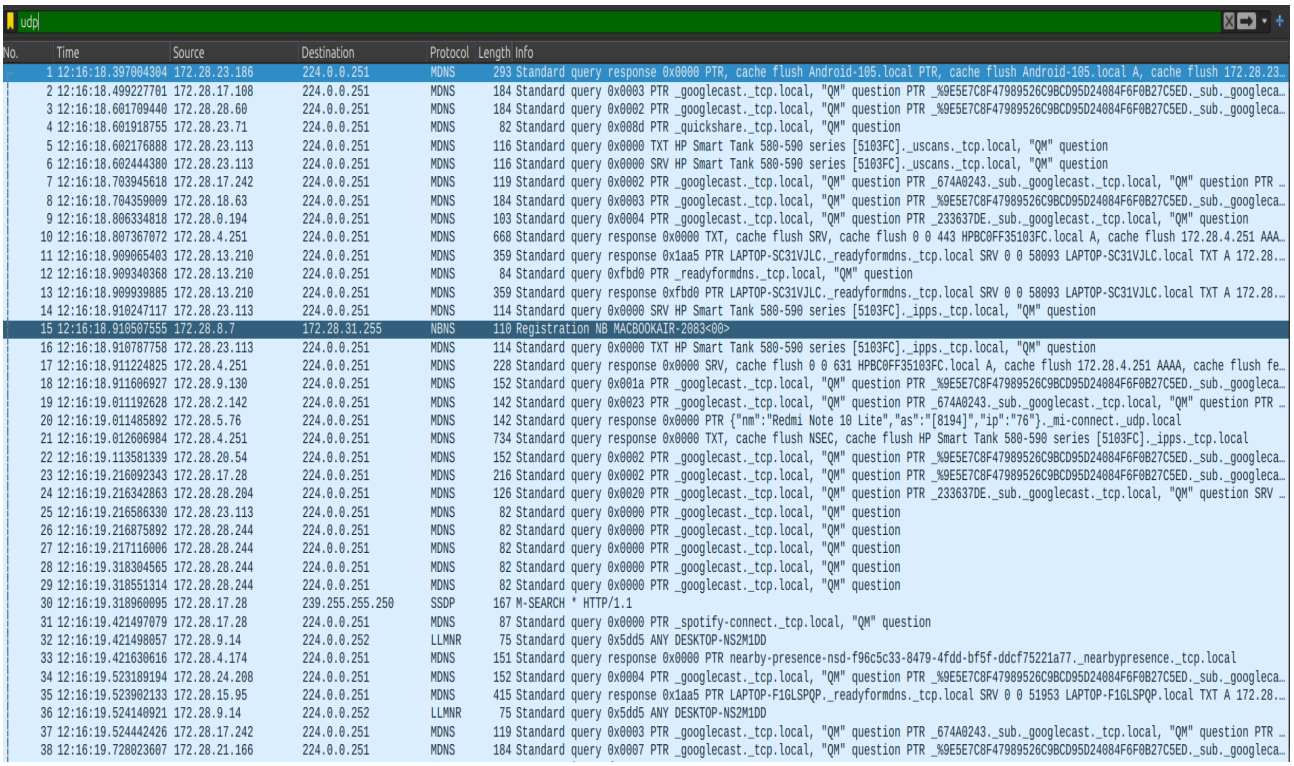
(a) HTTP display filter (`http`).



(b) TCP display filter (`tcp`).



(a) DNS display filter (`dns`).



(b) UDP display filter (`udp`).

Figure 8: Wireshark display filters isolating HTTP, TCP, DNS, and UDP traffic streams.

6.3 Part 5a: Following Network Conversations (UDP Stream Reassembly)

To inspect the end-to-end conversation flow, a datagram packet was selected, followed by **Right Click → Follow → UDP Stream**. Wireshark reassembles the fragmented payload bytes, strips the lower headers, and presents the unencapsulated application payload in a dedicated viewer window.

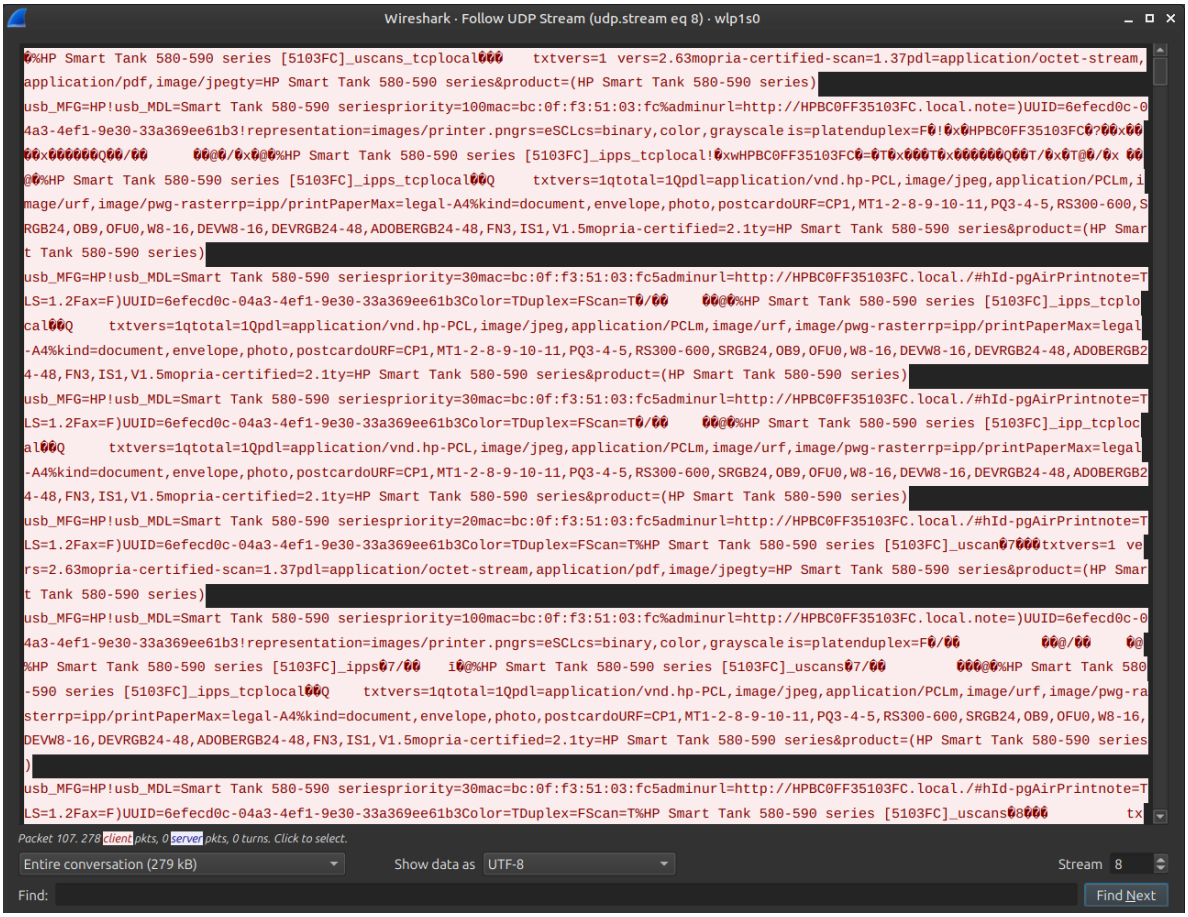


Figure 9: Wireshark stream reconstruction dialog for UDP Stream 8 (`udp.stream eq 8`) capturing an active mDNS service announcement.

6.3.1 Analysis of Stream 8 Payload

The reassembled UDP conversation captures high-frequency **Multicast DNS (mDNS)** broadcast records sent over multicast port `5353`. The packet contains zero-configuration service discovery advertisements broadcast by a networked printer:

- **Advertised Device Model:** `HP Smart Tank 580-590 series [5103FC]`
- **Network Hostname:** `HPBC0FF35103FC.local.`
- **Hardware MAC Address:** `bc:0f:f3:51:03:fc`
- **Discovered Network Services:**
  - `_uscans._tcp.local` (eSCL network scanning protocol)
  - `_ipps._tcp.local` (Secure Internet Printing Protocol)
  - `_ipp._tcp.local` (Standard Internet Printing Protocol)
- **Device Configuration Attributes:**
  - Admin URL: `http://HPBC0FF35103FC.local./#hId-pgAirPrint`
  - Device UUID: `6efecd0c-04a3-4ef1-9e30-33a369ee61b3`
  - Capabilities: `Color=T, Duplex=F, Scan=T`
  - Supported Page Description Languages: `application/octet-stream`, `application/pdf`, `image/jpeg`, `application/vnd.hp-PCL`, `image/urf`

This demonstrates how Wireshark's flow-following capability enables forensic extraction of cleartext application parameters from live network streams.

7 Question 6: Server HTTP Response and Ethernet Layer Inspection

7.1 Problem Statement

Search through your capture, and find an HTTP packet coming back from the server (TCP Source Port == 80). Expand the Ethernet layer in the Packet Details Panel.

7.2 Filtering and Ethernet Layer Dissection

To isolate HTTP response frames arriving from the web server, the combined display filter was executed:

```
http and tcp.srcport == 80
```

Packet No. 224 was selected, and the **Ethernet II** layer was expanded.

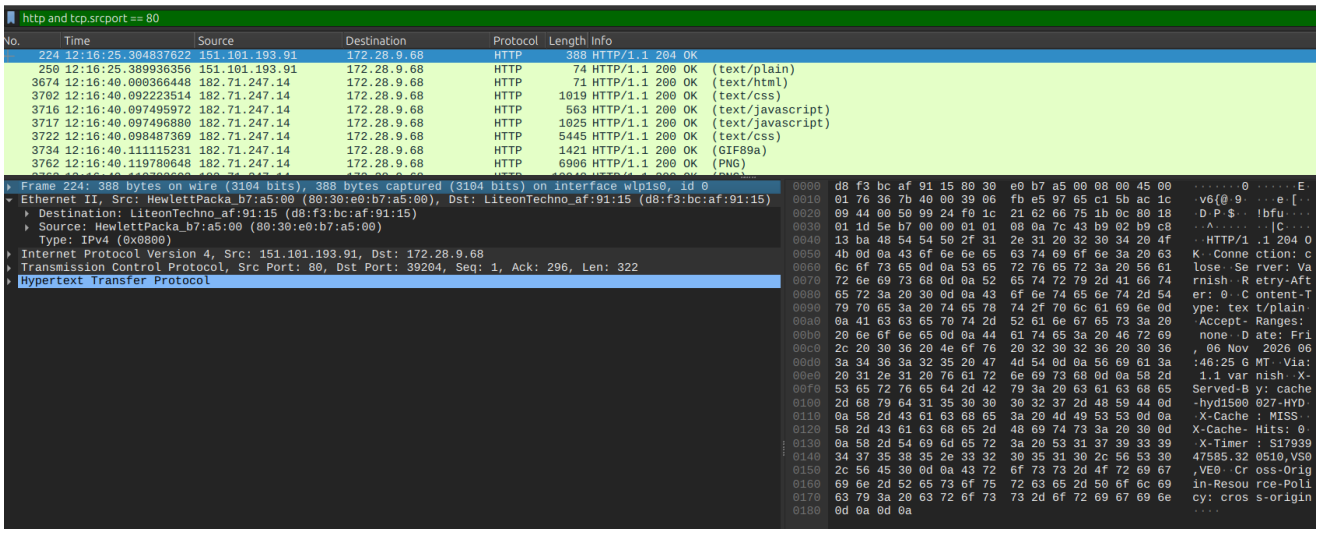


Figure 10: Wireshark display showing Packet 224 matching filter `http and tcp.srcport == 80` with expanded Ethernet II layer.

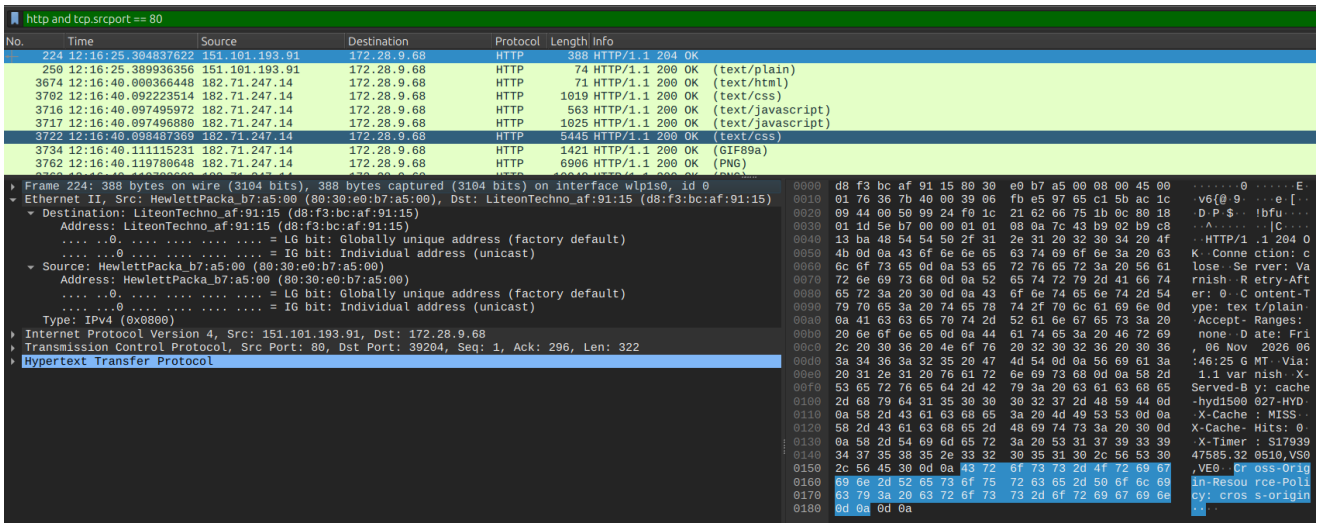


Figure 11: Detailed bit-level inspection of Destination and Source addresses in the Ethernet II header.

7.3 Structural Breakdown of Ethernet II Frame (Packet 224)

- **Frame Information:** 388 bytes on wire (3,104 bits) captured on interface `wlp1s0`.
- **Destination MAC Address (Recipient PC):** `LiteonTechno_af:91:15 (d8:f3:bc:af:91:15)`
  - `....  ..0.  ....` = LG bit: Globally unique address (factory default IEEE assigned)
  - `....  ...0  ....` = IG bit: Individual address (unicast frame)
- **Source MAC Address (Sending Node):** `HewlettPacka_b7:a5:00 (80:30:e0:b7:a5:00)`
  - `....  ..0.  ....` = LG bit: Globally unique address (factory default IEEE assigned)
  - `....  ...0  ....` = IG bit: Individual address (unicast frame)
- **EtherType Field:** `IPv4 (0x0800)` (indicates that payload contains an IPv4 packet).

8 Question 7: NIC Manufacturers of PC and Server

8.1 Problem Statement

What are the manufacturers of your PC's Network Interface Card (NIC), and the servers NIC?

8.2 Manufacturer Identification and Networking Principles

Referring to the expanded Ethernet II header in Figures 10 and 11:

1. **PC's Network Interface Card (NIC):**
   - **MAC Address:** `d8:f3:bc:af:91:15`
   - **Resolved Manufacturer Name: Lite-On Technology Corp.** (`LiteonTechno`)
   - *Physical Context:* This corresponds to the physical internal PCIe/M.2 Wi-Fi card (Lite-On / Qualcomm Atheros wireless module) inside the student's laptop operating interface `wlp1s0`.
2. **Server's / Gateway's Network Interface Card (NIC):**
   - **MAC Address:** `80:30:e0:b7:a5:00`
   - **Resolved Manufacturer Name: Hewlett Packard** (`HewlettPacka`)

8.2.1 Critical Theoretical Insight: Hop-by-Hop MAC Scope

In computer networking, it is crucial to recognize that **Layer-2 Ethernet MAC addresses never cross IP router boundaries**:

- The web server hosting `firefox-portal-detection.com` is located at IP address `151.101.193.91` on a remote Fastly CDN datacenter thousands of kilometers away.
- The original Ethernet frame transmitted by the remote server is decapsulated and re-encapsulated at every intermediate router along the Internet path.
- The frame arriving at the PC was placed onto the local wireless LAN by the **local default gateway router / wireless access point** (manufactured by **Hewlett Packard**).
- Therefore, while Wireshark displays **Hewlett Packard** as the Source MAC vendor of this incoming frame, this represents the **last-hop router/gateway NIC**, not the physical NIC of the remote web server itself.

9 Question 8: Organizationally Unique Identifiers (OUIs) in Raw Hex Bytes

9.1 Problem Statement

What are the Hex values (shown in the raw bytes panel) of the two NICS Manufacturers OUIs?

9.2 Theoretical Background

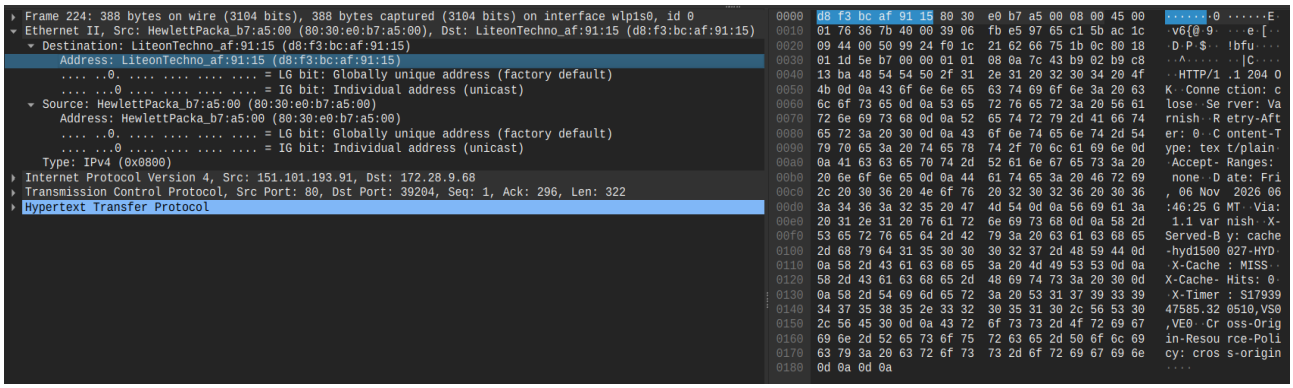
A standard IEEE 802 48-bit MAC address is partitioned into two 24-bit (3-byte) halves:

$$\underbrace{B_0 \quad B_1 \quad B_2}_{\textbf{OUI (24 bits) - Vendor ID}} \qquad \underbrace{B_3 \quad B_4 \quad B_5}_{\textbf{NIC Specific (24 bits) - Extension Identifier}} \tag{3}$$

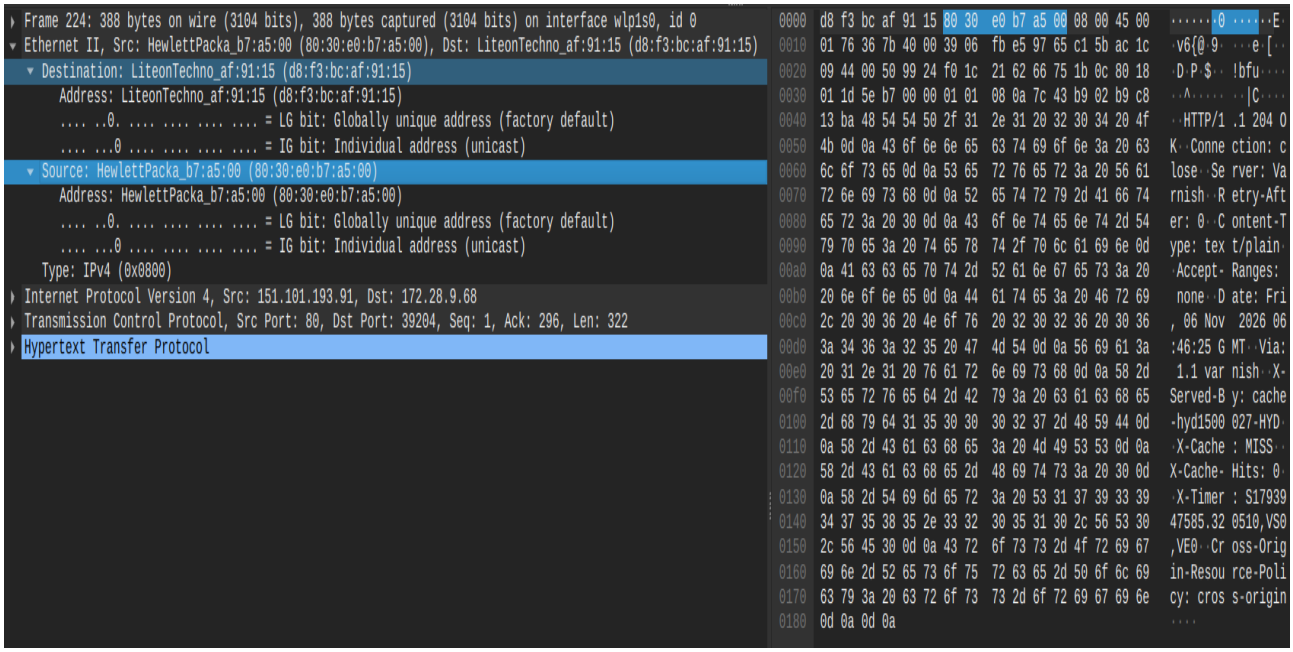
The **Organizationally Unique Identifier (OUI)** is assigned by the IEEE Registration Authority to hardware vendors, uniquely identifying the equipment manufacturer worldwide.

9.3 Inspection in Packet Bytes Panel

By selecting the Destination and Source address fields in Packet 224, the exact raw bytes are highlighted in the Packet Bytes Panel:



(a) PC NIC MAC Address `d8 f3 bc af 91 15` highlighted in raw bytes panel.



(a) Gateway / Server NIC MAC Address `80 30 e0 b7 a5 00` highlighted in raw bytes panel.

Figure 13: Extraction of vendor OUIs from raw hexadecimal packet bytes.

9.4 Extracted OUI Hex Values

Interface / Node	Full MAC Address	OUI (Hex)	OUI (Bytes)	IEEE Assigned Vendor
PC Client NIC	`d8:f3:bc:af:91:15`	`d8:f3:bc`	`0xd8, 0xf3, 0xbc`	Lite-On Technology Corp.
Gateway / Server NIC	`80:30:e0:b7:a5:00`	`80:30:e0`	`0x80, 0x30, 0xe0`	Hewlett Packard Enterprise

Table 3: Hexadecimal values of the two manufacturer OUIs from the raw packet bytes.

Answers:

- Hex value of PC NIC Manufacturer OUI: `d8:f3:bc` (or `d8 f3 bc`)
- Hex value of Gateway / Server NIC Manufacturer OUI: `80:30:e0` (or `80 30 e0`)

10 Question 9: Protocol Hierarchy Statistics

10.1 Problem Statement

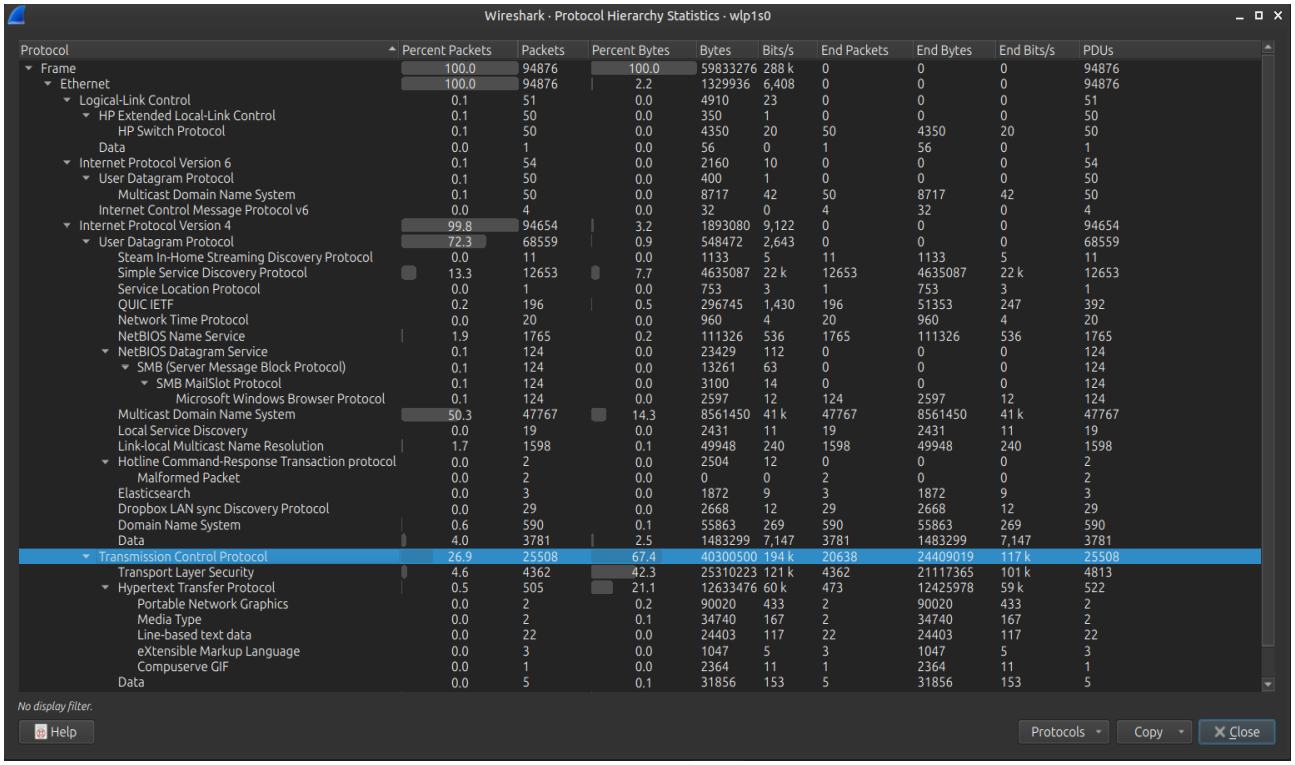
Find the following statistics:

a. *What percentage of packets in your capture are TCP, and give an example of the higher level protocol which uses TCP?*

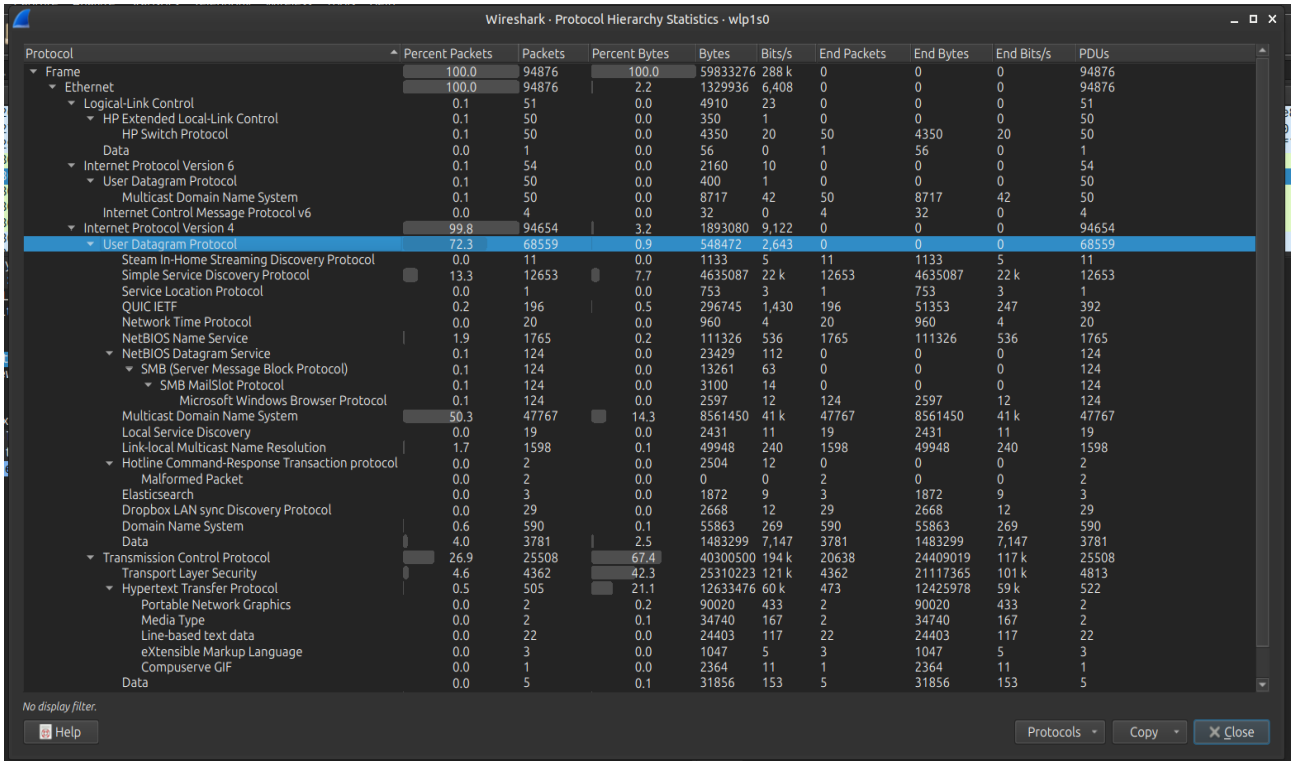
b. *What percentage of packets in your capture are UDP, and give an example of the higher level protocol which uses UDP?*

10.2 Aggregate Capture Hierarchy Profiling

Opening **Statistics** → **Protocol Hierarchy** generates an aggregated statistical profile of the complete 94,876 captured packets (totaling 59,833,276 bytes).



(a) Protocol hierarchy statistics highlighting Transmission Control Protocol (TCP).



(b) Protocol hierarchy statistics highlighting User Datagram Protocol (UDP).

Figure 14: Wireshark Protocol Hierarchy Statistics window for the complete capture session.

10.3 Part 9a: TCP Statistics and Application Protocols

From Figure 14a:

- **Percentage of Packets that are TCP:** 26.9% (25,508 packets out of 94,876).
- **Byte Volume:** 40,300,500 bytes (**67.4%** of all captured byte traffic).
- **Examples of Higher-Level Protocols Using TCP:**
  - **Transport Layer Security (TLS):** 4,362 packets (**4.6%** of total packets; 25,310,223 bytes).

- **Hypertext Transfer Protocol (HTTP):** 505 packets (**0.5%** of total packets; 12,633,476 bytes).
- Subordinate media payloads transported over HTTP: Portable Network Graphics (PNG), Extensible Markup Language (XML), and Cascading Style Sheets (CSS).

10.4 Part 9b: UDP Statistics and Application Protocols

From Figure 14b:

- **Percentage of Packets that are UDP:** `72.3%` (68,559 packets out of 94,876).
- **Byte Volume:** 548,472 bytes (**0.9%** of all captured byte traffic).
- **Examples of Higher-Level Protocols Using UDP:**
  - **Multicast Domain Name System (mDNS):** 47,767 packets (**50.3%** of total capture packets).
  - **Simple Service Discovery Protocol (SSDP):** 12,653 packets (**13.3%** of total packets).
  - **NetBIOS Name Service (NBNS):** 1,765 packets (**1.9%** of total packets).
  - **Link-Local Multicast Name Resolution (LLMNR):** 1,598 packets (**1.7%** of total packets).
  - **Domain Name System (DNS):** 590 packets (**0.6%** of total packets).
  - **QUIC IETF:** 196 packets (**0.2%** of total packets).
  - **Network Time Protocol (NTP):** 20 packets ($< 0.1\%$).

10.5 Comparative Analysis: Packet Count vs. Byte Volume Asymmetry

Protocol	Packets	% of Packets	Bytes	% of Bytes	Avg Size	Primary Function in Trace
UDP	68,559	**72.3%**	548,472	**0.9%**	≈ 8 B	High-frequency lightweight multicast discovery and beaconing.
TCP	25,508	**26.9%**	40,300,500	**67.4%**	≈ 1580 B	Heavyweight reliable web browsing, TLS, and media transfers.
Other / LLC	809	0.8%	18,984,304	31.7%	—	Link-layer control, IPv6, HP switch protocols.

Table 4: Comparative analysis of transport layer protocol distribution.

The data highlights a classic networking asymmetry: UDP dominates **packet count** (72.3%) due to rapid zero-configuration multicast discovery bursts, yet TCP dominates **byte throughput** (67.4%) because bulky web assets and TLS packets fill full Ethernet MTUs (1500 bytes).

11 Question 10: Traffic Flow Graph Analysis

11.1 Problem Statement

Find the traffic flow. Select the Statistics → Flow Graph menu option. Choose General Flow and Network Source options, and click the OK button.

11.2 Overview of Wireshark Flow Graph

The **Flow Graph** feature renders a two-dimensional ladder (sequence) diagram visualizing network communications over time. Communicating endpoints are mapped as vertical timeline axes, and packets are plotted as horizontal/diagonal arrows indicating directional flow, sequence numbers, acknowledgment numbers, and payload flags.

11.3 Step 1: General Flow Graph (All Flows, Network Source)

Selecting **Flow type: All Flows** with **Addresses: Network** generates the holistic traffic interaction graph across all subnet IP nodes.

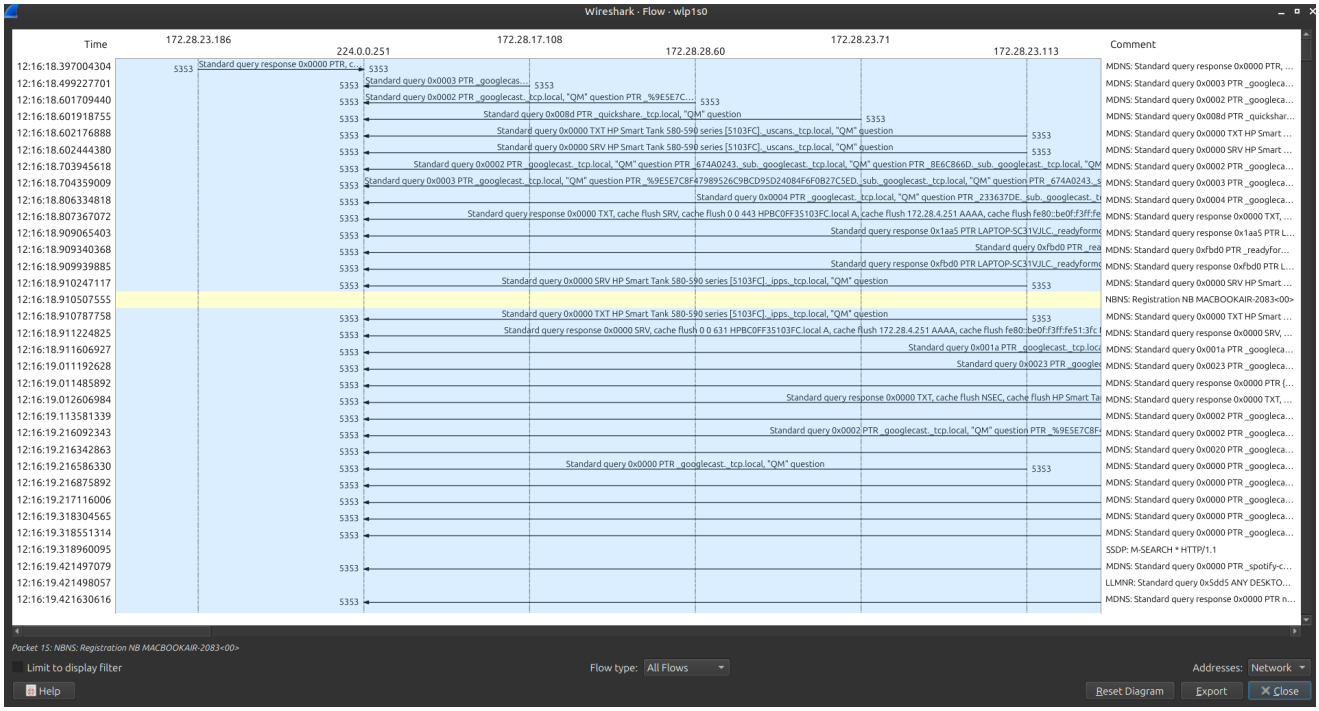


Figure 15: Wireshark General Flow Graph (All Flows, Network Addresses) showing multicast discovery traffic.

11.3.1 General Flow Observations:

- Multiple local IP endpoints (`172.28.23.186`, `172.28.17.108`, `172.28.28.60`, `172.28.23.71`, `172.28.23.113`) interact with the standard mDNS multicast IP `224.0.0.251`.
- Simultaneous NetBIOS Name Service broadcast registration requests (for example, host `MACBOOKAIR-2083`) demonstrate continuous background host and service discovery across the campus subnet.

11.4 Step 2: TCP Flow Graph (TCP Connection State Machine)

Selecting **Flow type: TCP Flows** isolates the state transitions and reliable byte-stream exchange between client `172.28.9.68` and web server `151.101.193.91`.



Figure 16: Wireshark TCP Flow Graph showing the complete TCP connection lifecycle (Handshake → Data Transfer → Teardown).

11.4.1 Detailed Breakdown of the TCP Flow Lifecycle

The TCP Flow graph reveals the three distinct phases of the connection:

1. **Phase 1: Three-Way Handshake (Connection Establishment):**
   - **Step 1 (t_1):** Client sends `SYN` packet (`39204` $\rightarrow$ `80`, `Seq=0`, `Len=0`) requesting connection setup.
   - **Step 2 (t_2):** Server replies with `SYN, ACK` (`80` $\rightarrow$ `39204`, `Seq=0`, `Ack=1`) confirming synchronization and acknowledging client ISN.
   - **Step 3 (t_3):** Client sends `ACK` (`39204` $\rightarrow$ `80`, `Seq=1`, `Ack=1`), establishing the `ESTABLISHED` state.
2. **Phase 2: HTTP Transaction (Data Exchange):**
   - **Step 4 (t_4):** Client pushes HTTP GET request: `PSH, ACK - Len: 295` (`Seq=1`, `Ack=1`).
   - **Step 5 (t_5):** Server acknowledges request receipt: `ACK` (`Seq=1`, `Ack=296`).
   - **Step 6 (t_6):** Server delivers HTTP 204 OK response: `PSH, ACK - Len: 322` (`Seq=1`, `Ack=296`).
3. **Phase 3: Connection Termination (Four-Way Teardown):**
   - **Step 7 (t_7):** Server initiates close: `FIN, ACK` (`Seq=323`, `Ack=296`).
   - **Step 8 (t_8):** Client acknowledges server close: `ACK` (`Seq=296`, `Ack=323`).
   - **Step 9 (t_9):** Client sends its own close: `FIN, ACK` (`Seq=296`, `Ack=324`).
   - **Step 10 (t_{10}):** Server completes termination: `ACK` (`Seq=324`, `Ack=297`).

12 Conclusion and Comments

12.1 Summary of Results

This laboratory assignment successfully demonstrated the end-to-end operation of network protocol stacks using Wireshark for deep packet inspection. Key findings across all 10 questions include:

1. **ARP & ICMP Synergy:** Verified that IP diagnostic tools (`ping`) depend strictly upon prior ARP broadcast resolution to establish Data Link Layer destination addresses before transmitting ICMP Echo Requests.
2. **Web Protocol Complexity:** Captured real-world web transactions comprising HTTP, TCP, DNS, TLS, QUIC, and ICMP, measuring an exact HTTP Round-Trip Time of **39.46** ms.
3. **Binary to Text Alignment:** Validated that the Host header `firefox-portal-detection.com` corresponds to raw bytes starting with `0x48 0x6f 0x73 0x74` (spelling `Host`).
4. **Stream Reassembly:** Followed UDP stream 8 to reconstruct cleartext mDNS device descriptors broadcast by an HP Smart Tank printer.
5. **Hardware Identification via OUIs:** Proved that the local client NIC (`d8:f3:bc`) was manufactured by Lite-On Technology Corp., whereas the incoming frame's source MAC (`80:30:e0`) identified the local default gateway router manufactured by Hewlett Packard.
6. **Traffic Asymmetry:** Demonstrated that UDP represented **72.3**% of total packets but only **0.9**% of bytes due to discovery broadcasts, while TCP accounted for **67.4**% of all byte throughput.
7. **Flow Visualization:** Visualized the complete 3-way handshake, payload exchange, and 4-way teardown using Wireshark Flow Graphs.

12.2 What I Learned

- Mastered live capture workflows and filtering syntax in Wireshark.
- Deepened understanding of the OSI/TCP-IP layering abstraction and packet encapsulation.
- Understood the critical distinction between hop-by-hop Layer-2 MAC rewriting and end-to-end Layer-3 IP addressing.
- Gained practical skills in analyzing protocol overhead, connection state machines, and transport layer performance.

References

[1] Kurose, J. F., and Ross, K. W., *Computer Networking: A Top-Down Approach*, 8th ed., Pearson, 2021.

[2] Wireshark Documentation Team, *Wireshark User's Guide*, v4.2, https://www.wireshark.org/docs/wsug_html_chunked/, 2026.

[3] Fielding, R., et al., *Hypertext Transfer Protocol – HTTP/1.1*, RFC 2616, Internet Engineering Task Force (IETF), 1999.

[4] IEEE Standards Association, *IEEE Standard for Ethernet*, IEEE Std 802.3-2022.